

LAPORAN PRAKTIKUM STRUKTUR DATA

DOUBLE LINKED LIST

disusun oleh :

Khairunnisa M.

NIM 2511532005

Dosen Pengampu :

Dr. Wahyudi,. S.T., M.T.

Asisten Dosen :

M. Ghalid



DEPARTEMEN INFORMATIKA
FAKULTAS TEKNOLOGI INFORMASI
UNIVERSITAS ANDALAS

2026

KATA PENGANTAR

Puji Syukur penulis panjatkan kehadiran Tuhan Yang Maha Esa atas rahmat dan karunia-Nya laporan praktikum Struktur Data dengan materi Double Linked List ini dapat diselesaikan dengan baik. Laporan ini disusun sebagai bentuk dokumentasi dan pemahaman penulis terhadap konsep Double Linked List dalam struktur data serta implementasinya menggunakan bahasa pemrograman Java.

Dalam laporan ini, penulis membahas beberapa implementasi Double Linked List, serta operasi – operasi dasar yang terdapat pada Double Linked List.

Penulis menyadari bahwa lapoaran ini masih memiliki kekurangan. Oleh karena itu, penulis mengharapkan kritik dan saran yang membangun untuk perbaikan di masa mendatang. Akhir kata, semeoga laporan ini dapat memberikan manfaat bagi pembaca.

Padang, 12 Mei 2026

Khairunnisa M.

DAFTAR ISI

KATA PENGANTAR.....	ii
DAFTAR ISI	iii
BAB I.....	1
PENDAHULUAN	1
1.1. Latar Belakang	1
1.2. Tujuan Praktikum	1
1.3. Manfaat Praktikum.....	1
BAB II	3
PEMBAHASAN.....	3
2.1. Deskripsi Materi Single Linked List.....	3
2.2. NodeSLL_2511532005.....	4
2.3. TambahSLL_2511532005.	4
2.4. PencarianSLL_2511532005.....	6
2.5. HapusSLL_2511532005.....	8
2.6. Analisis Praktikum.....	9
BAB III.....	11
KESIMPULAN DAN SARAN.....	11
3.1 Kesimpulan.....	11
3.2 Saran.....	11
DAFTAR PUSTAKA	12

BAB I

PENDAHULUAN

1.1. Latar Belakang

Struktur data merupakan salah satu konsep penting dalam pemrograman yang digunakan untuk mengatur dan menyimpan data agar dapat diakses secara efisien. Salah satu jenis struktur data liner yang sering digunakan adalah Linked List. Dalam penggunaannya, Linked List memiliki beberapa jenis, salah satunya yaitu Double Linked List yang memungkinkan data diakses dari dua arah.

Double Linked List memiliki kelebihan karena setiap node memiliki dua pointer, yaitu *next* dan *prev*, sehingga proses penelusuran data menjadi lebih fleksibel. Oleh karena itu, praktikum ini dilakukan agar mahasiswa memahami konsep dasar Double Linked List serta mampu mengimplementasikannya menggunakan bahasa pemrograman Java.

1.2. Tujuan Praktikum

1. Memahami konsep dasar *Double Linked List*.
2. Mengetahui struktur node pada *Double Linked List*.
3. Mengimplementasikan *Double Linked List* menggunakan bahasa Java.
4. Memahami penggunaan pointer *next* dan *prev*.
5. Melatih kemampuan pemrograman dalam struktur data dinamis

1.3. Manfaat Praktikum

1. Mahasiswa dapat memahami konsep struktur data Double Linked List.
2. Mahasiswa mampu membuat program Double Linked List menggunakan Java.
3. Mahasiswa memahami hubungan antar node dalam struktur data.

4. Mahasiswa dapat menerapkan konsep struktur data dalam pengembangan program yang lebih kompleks.
5. Menambah kemampuan logika pemrograman dan analisis data.

BAB II

PEMBAHASAN

2.1. Deskripsi Materi Double Linked List

Struktur Data Double Linked List adalah struktur data linear yang terdiri dari kumpulan node yang saling terhubung. Setiap node menyimpan data dan buah pointer, yaitu :

- a. next = menunjuk ke node berikutnya.
- b. prev = menunjuk ke node sebelumnya.

Berbeda dengan Single Linked List yang hanya memiliki satu arah transversal, Double Linked List memungkinkan penelusuran dilakukan dari depan ke belakang maupun dari belakang ke depan

Karakteristik Double Linked List :

1. setiap node memiliki dua pointer.
2. Traversal dapat dilakukan dua arah.
3. Penyisipan dan penghapusan node lebih fleksibel.
4. Membutuhkan memori lebih banyak dibanding Single Linked List.

Kelebihan Double Linked List :

1. Dapat menelusuri data dua arah.
2. Mempermudah proses penghapusan node.
3. Lebih fleksibel dalam manipulasi data.

Kekurangan Double Linked List :

1. Membutuhkan memori lebih besar.
2. Implementasi lebih kompleks dibanding Single Linked List.

2.2. NodeDLL_2511532005

Program di atas merupakan kelas node untuk implementasi Double Linked List dalam bahasa Java. Kelas ini digunakan untuk menyimpan data dan menghubungkan node satu dengan node lainnya.

```

1 package pekan6_2511532005;
2
3 public class NodeDLL_2511532005 {
4     // mengidentifikasikan kelas Node
5     int data_2005; // data
6     NodeDLL_2511532005 next_2005; // pointer ke next node
7     NodeDLL_2511532005 prev_2005; // pointer ke previous node
8
9     // konstruktor
10    public NodeDLL_2511532005 (int data_2005) {
11        this.data_2005 = data_2005;
12        this.next_2005 = null;
13        this.prev_2005 = null;
14    }
15
16
17 }
18

```

Gambar 2.1. Kode program NodeDLL_2511532005.

Kode program dan fungsi nya :

1. next_2005 = berfungsi sebagai pointer yang menunjuk ke node berikutnya.
2. prev_2005 = digunakan untuk menunjuk ke node sebelumnya.
3. Konstruktor digunakan untuk menginisialisasi objek node saat pertama kali dibuat.

2.3. InsertDLL_2511532005.

Kode program ini digunakan untuk menambahkan node pada struktur data Double Linked List program ini memiliki beberapa fungsi yaitu menambah node di awal, di akhir, dan pada posisi tertentu.

```

1 package pekan6_2511532005;
2
3 public class InsertDLL_2511532005 {
4     // menambahkan node di awal DLL
5     static NodeDLL_2511532005 insertBegin (NodeDLL_2511532005 head_2005, int data_2005) {
6         NodeDLL_2511532005 new_node = new NodeDLL_2511532005(data_2005); // buat node baru
7         new_node.next_2005 = head_2005; // jadikan pointer next nya head
8         if (head_2005 != null) {
9             head_2005.prev_2005 = new_node;
10        }
11        return new_node;
12    }
13    // fungsi menambahkan node di akhir
14    public static NodeDLL_2511532005 insertEnd (NodeDLL_2511532005 head_2005, int newData_2005) {
15        // buat node baru
16        NodeDLL_2511532005 newNode = new NodeDLL_2511532005(newData_2005);
17        // jika di null jadikan head
18        if (head_2005 == null) {
19            head_2005 = newNode;
20        }
21        else {
22            NodeDLL_2511532005 curr_2005 = head_2005;
23            while (curr_2005.next_2005 != null) {
24                curr_2005 = curr_2005.next_2005;
25            }
26            curr_2005.next_2005 = newNode;
27            newNode.prev_2005 = curr_2005;
28        }
29        return head_2005;
30    }

```

```

32 //fungsi menambahkan node di posisi teratas
33 public static NodeDLL_2511532005 insertAtPosition (NodeDLL_2511532005 head_2005, int pos_2005, int new_data) {
34     //buat node baru
35     NodeDLL_2511532005 new_node = new NodeDLL_2511532005 (new_data);
36     if (pos_2005 == 1) {
37         new_node.next_2005 = head_2005;
38         if (head_2005 != null) {
39             head_2005.prev_2005 = new_node;
40         }
41         head_2005 = new_node;
42         return head_2005;
43     }
44     NodeDLL_2511532005 curr_2005 = head_2005;
45     for (int i = 1; i < pos_2005 - 1 && curr_2005 != null; ++i) {
46         curr_2005 = curr_2005.next_2005;
47     }
48     if (curr_2005 == null) {
49         System.out.println ("Posisi tidak ada");
50         return head_2005;
51     }
52     new_node.prev_2005 = curr_2005;
53     new_node.next_2005 = curr_2005.next_2005;
54     curr_2005.next_2005 = new_node;
55     if (new_node.next_2005 != null) {
56         new_node.next_2005.prev_2005 = new_node;
57     }
58     return head_2005;
59 }

```

```

60 public static void printList (NodeDLL_2511532005 head_2005) {
61     NodeDLL_2511532005 curr_2005 = head_2005;
62     while (curr_2005 != null) {
63         System.out.print (curr_2005.data_2005 + " <-> ");
64         curr_2005 = curr_2005.next_2005;
65     }
66     System.out.println ();
67 }
68
69 public static void main (String[] args) {
70     //membuat dll 2 <-> 3 <-> 5
71     NodeDLL_2511532005 head_2005 = new NodeDLL_2511532005 (2);
72     head_2005.next_2005 = new NodeDLL_2511532005 (3);
73     head_2005.next_2005.prev_2005 = head_2005;
74     head_2005.next_2005.next_2005 = new NodeDLL_2511532005 (5);
75     head_2005.next_2005.next_2005.prev_2005 = head_2005.next_2005;
76     //cetak DLL awal
77     System.out.print ("DLL Awal: ");
78     printList (head_2005);
79     //tambah 1 di awal
80     head_2005 = insertBegin (head_2005, 1);
81     System.out.print ("Simpul 1 ditambah di awal: ");
82     printList (head_2005);
83     //tambah 6 di akhir
84     System.out.print ("Simpul 6 ditambah di akhir: ");
85     int data_2005 = 6;
86     head_2005 = insertEnd (head_2005, data_2005);
87     printList (head_2005);
88     //menambahkan node 4 di posisi 4
89     System.out.print ("Tambah node 4 di posisi 4: ");
90
91     System.out.print ("Tambah node 4 di posisi 4: ");
92     int data2_2005 = 4;
93     int pos_2005 = 4;
94     head_2005 = insertAtPosition (head_2005, pos_2005, data2_2005);
95     printList (head_2005);
96 }

```

Gambar 2.2. Kode Program InerDLL_2511532005.

Kode program dan fungsinya :

1. insertBegin() = digunakan untuk menambahkan node di awal Double Linked List. Node baru akan menjadi head baru dan pointer prev pada node lama akan diarahkan ke node baru.
2. insertEnd() = digunakan untuk menambahkan node di akhir list. Program akan menelusuri node sampai node terakhir, kemudian menghubungkan node baru di bagian akhir.
3. insertAtPosition() = digunakan untuk menambahkan node pada posisi tertentu. Node baru akan disisipkan diantara node sebelumnya.

2.4. PenelusuranDLL_2511532005

Program ini digunakan untuk melakukan penelusuran (traversal) pada struktur data Double Linked List baik secara maju maupun mundur.

```

1 package pekan6_2511532005;
2
3 public class PenelusuranDLL_2511532005 {
4     //fungsi penelusuran maju
5     static void forwardTraversal (NodeDLL_2511532005 head_2005) {
6         //memulai penelusuran dari head
7         NodeDLL_2511532005 curr_2005 = head_2005;
8         //lanjutkan sampai akhir
9         while (curr_2005 != null) {
10             //print data
11             System.out.print (curr_2005.data_2005 + " <-> ");
12             //pindahkan ke node berikutnya
13             curr_2005 = curr_2005.next_2005;
14         }
15         //print spasi
16         System.out.println ();
17     }
18     //fungsi penelusuran mundur
19     static void backwardTraversal (NodeDLL_2511532005 tail_2005) {
20         //mulai dari akhir
21         NodeDLL_2511532005 curr_2005 = tail_2005;
22         //lanjut sampai head
23         while (curr_2005 != null) {
24             //cetak data
25             System.out.print (curr_2005.data_2005 + " <-> ");
26             //pindahkan ke node sebelumnya
27             curr_2005 = curr_2005.prev_2005;
28         }
29         //cetak spasi
30         System.out.println();
31     }
32
33     public static void main(String[] args) {
34         // cetak DLL
35         NodeDLL_2511532005 head_2005 = new NodeDLL_2511532005(1);
36         NodeDLL_2511532005 second_2005 = new NodeDLL_2511532005(2);
37         NodeDLL_2511532005 third_2005 = new NodeDLL_2511532005(3);
38
39         head_2005.next_2005 = second_2005;
40         second_2005.prev_2005 = head_2005;
41         second_2005.next_2005 = third_2005;
42         third_2005.prev_2005 = second_2005;
43
44         System.out.println("penelusuran maju :");
45         forwardTraversal(head_2005);
46
47         System.out.println("penelusuran mundur :");
48         backwardTraversal(third_2005);
49     }
50 }

```

Gambar 2.3. Kode Program PenelusuranDLL_2511532005

Kode dan fungsi :

1. forwardTraversal() = digunakan untuk menelusuri data dari depan ke belakang dimulai dari head_2005. Penelusuran dilakukan menggunakan pointer next_2005.
2. backwardTraversal() = digunakan untuk menelusuri data dari belakang ke depan dimulai dari node terakhir. Penelusuran dilakukan menggunakan pointer prev_2005.

2.5. HapusSLL_2511532005

Program ini digunakan untuk menghapus node pada struktur data Double Linked List. Program ini dapat menghapus node di awal, di akhir, dan pada posisi tertentu.

```
package pekan6_2511532005;

public class HapusDLL_2511532005 {
    //FUNGSI menghapus node awal
    public static NodeDLL_2511532005 delHead_2005 (NodeDLL_2511532005 head_2005) {
        if (head_2005 == null) {
            return null;
        }
        NodeDLL_2511532005 temp = head_2005;
        head_2005 = head_2005.next_2005;
        if (head_2005 != null) {
            head_2005.prev_2005 = null;
        }
        return head_2005;
    }
    // fungsi menghapus akhir
    public static NodeDLL_2511532005 dellast_2005 (NodeDLL_2511532005 head_2005) {
        if (head_2005 == null) {
            return null;
        }
        if (head_2005.next_2005 == null) {
            return null;
        }
        NodeDLL_2511532005 curr_2005 = head_2005;
        while (curr_2005.next_2005 != null) {
            curr_2005 = curr_2005.next_2005;
        }
        // update pointer previous node
        if (curr_2005.prev_2005 != null) {
            curr_2005.prev_2005.next_2005 = null;
        }
    }
}
```

```

32     return head_2005;
33 }
34 // fungsi menghapus node posisi tertentu
35 public static NodeDLL_2511532005 delPos_2005 (NodeDLL_2511532005 head_2005, int pos_2005) {
36     // jika DLL kosong
37     if (head_2005 == null) {
38         return head_2005;
39     }
40     NodeDLL_2511532005 curr_2005 = head_2005;
41     // telusuri sampai ke node yang akan di hapus
42     for (int i = 1; curr_2005 != null && i < pos_2005; ++i) {
43         curr_2005 = curr_2005.next_2005;
44     }
45     // jika posisi tidak di temukan
46     if (curr_2005 == null) {
47         return head_2005;
48     }
49     // update pointer
50     if (curr_2005.prev_2005 != null) {
51         curr_2005.prev_2005.next_2005 = curr_2005.next_2005;
52     }
53     if (curr_2005.next_2005 != null) {
54         curr_2005.next_2005.prev_2005 = curr_2005.prev_2005;
55     }
56     // jika yang dihapus head
57     if (head_2005 == curr_2005) {
58         head_2005 = curr_2005.next_2005;
59     }
60     return head_2005;
61 }

```

```

62 // fungsi mencetak DLL
63 public static void printList(NodeDLL_2511532005 head_2005) {
64     NodeDLL_2511532005 curr_2005 = head_2005;
65     while (curr_2005 != null) {
66         System.out.print(curr_2005.data_2005 + " ");
67         curr_2005 = curr_2005.next_2005;
68     }
69     System.out.println();
70 }
71 public static void main(String[] args) {
72     // buat sebuah DLL
73     NodeDLL_2511532005 head_2005 = new NodeDLL_2511532005(1);
74     head_2005.next_2005 = new NodeDLL_2511532005(2);
75     head_2005.next_2005.prev_2005 = head_2005;
76     head_2005.next_2005.next_2005 = new NodeDLL_2511532005(3);
77     head_2005.next_2005.next_2005.prev_2005 = head_2005.next_2005;
78     head_2005.next_2005.next_2005.next_2005 = new NodeDLL_2511532005(4);
79     head_2005.next_2005.next_2005.next_2005.prev_2005 = head_2005.next_2005.next_2005;
80     head_2005.next_2005.next_2005.next_2005.next_2005 = new NodeDLL_2511532005(5);
81     head_2005.next_2005.next_2005.next_2005.next_2005.prev_2005 = head_2005.next_2005.next_2005.next_2005;
82
83     System.out.print("DLL Awal: ");
84     printList(head_2005);
85
86     System.out.print("Setelah head dihapus: ");
87     head_2005 = delHead_2005(head_2005);
88     printList(head_2005);
89
90     System.out.print("Setelah node terakhir dihapus: ");
91     head_2005 = delLast_2005(head_2005);
92     printList(head_2005);
93
94     System.out.print("menghapus node ke 2: ");
95     head_2005 = delPos_2005(head_2005, 2);
96
97     printList(head_2005);
98 }
99 }
100

```

Gambar 2.4. Kode Porgram HapusDLL_2511532005.

Kode dan fungsi :

1. `delHead_2005()` = digunakan untuk menghapus node pertama pada double linked list. Setelah node pertama dihapus, head akan berpindah ke node berikutnya.
2. `delLast_2005()` = digunakan untuk menghapus node terakhir pada list. Program akan menelusuri hingga node terakhir lalu memutus hubungan dengan node sebelumnya.
3. `DelPos_2005()` = digunakan untuk menghapus node pada posisi tertentu. Pointer next dan prev diperbarui agar hubungan antar node tetap tersambung.

2.6. Analisis Praktikum

Pada praktikum ini telah dilakukan implementasi struktur data Double Linked List menggunakan Bahasa pemrograman Java. Praktikum mencakup

pembuatan node, proses penambahan node di awal, akhir, dan posisi tertentu, penelusuran data secara maju dan mundur, serta penghapusan node pada beberapa posisi. Dari praktikum ini dapat dipahami bahwa setiap node memiliki dua pointer yaitu next dan prev yang berfungsi untuk menghubungkan node dengan node berikutnya dan sebelumnya.

Hasil praktikum menunjukkan bahwa *Double Linked List* memiliki kelebihan karena data dapat ditelusuri dalam dua arah sehingga proses manipulasi data menjadi lebih mudah dan fleksibel. Selain itu, operasi penambahan dan penghapusan node dapat dilakukan dengan baik melalui pengaturan pointer antar node. Namun, struktur data ini membutuhkan memori lebih banyak dibandingkan *Single Linked List* karena setiap node memiliki dua pointer.

BAB III

KESIMPULAN DAN SARAN

3.1 Kesimpulan

Berdasarkan hasil praktikum yang telah dilakukan, dapat disimpulkan bahwa *Double Linked List* merupakan struktur data linear yang setiap node-nya memiliki dua pointer, yaitu next dan prev. Dengan adanya dua pointer tersebut, proses penelusuran data dapat dilakukan dari depan maupun dari belakang sehingga lebih fleksibel dalam pengolahan data.

Melalui praktikum ini, mahasiswa dapat memahami cara membuat node, menambahkan node, melakukan traversal, serta menghapus node pada *Double Linked List* menggunakan bahasa pemrograman Java. Praktikum ini juga membantu meningkatkan pemahaman mengenai konsep struktur data dinamis dan hubungan antar node.

3.2 Saran

Dalam melakukan praktikum struktur data, diperlukan ketelitian dalam mengatur pointer next dan prev agar hubungan antar node tidak mengalami kesalahan. Selain itu, mahasiswa disarankan untuk lebih sering berlatih membuat program *Double Linked List* agar semakin memahami proses manipulasi data pada struktur data tersebut.

Untuk pengembangan selanjutnya, praktikum dapat diperluas dengan implementasi operasi lain seperti pencarian data, sorting, maupun penerapan *Double Linked List* pada kasus yang lebih kompleks.

DAFTAR PUSTAKA

Khan, M. Z. (2023). *Introduction to Doubly Linked Lists in Java*. Retrieved from GeeksForGeeks.
Linked list Data Structure. (n.d.). Retrieved from Progamiz:
<https://www.programiz.com/dsa/linked-list>

TUGAS PRAKTIKUM 6

Playlist Lagu

1. Perintah Tugas

Buatlah program sederhana untuk mengelola playlist lagu menggunakan Double Linked List. Setiap lagu adalah node yang memiliki pointer ke lagu sebelumnya (prev) dan lagu berikutnya (next). Program harus memungkinkan pengguna menambah lagu, menghapus lagu, dan melihat daftar lagu dari dua arah.

2. ADT Lagu

```

1 package pekan6_2511532005;
2
3 public class Lagu_2511532005 {
4     private String judul_2005;
5     private String penyanyi_2005;
6     private Lagu_2511532005 next_2005; // Pointer ke lagu berikutnya
7     private Lagu_2511532005 prev_2005; // Pointer ke lagu sebelumnya (WAJIB DLL)
8     // Constructor
9     public Lagu_2511532005(String judul_2005, String penyanyi_2005) {
10         this.judul_2005 = judul_2005;
11         this.penyanyi_2005 = penyanyi_2005;
12         this.next_2005 = null;
13         this.prev_2005 = null;
14     }
15     // Getter methods
16     public String getJudul_2005() {return judul_2005;}
17     public String getPenyanyi_2005() {return penyanyi_2005;}
18     public Lagu_2511532005 getNext_2005() {return next_2005;}
19     public Lagu_2511532005 getPrev_2005() {return prev_2005;}
20
21     // Setter methods
22     public void setNext_2005(Lagu_2511532005 next_2005) {this.next_2005 = next_2005;}
23     public void setPrev_2005(Lagu_2511532005 prev_2005) {this.prev_2005 = prev_2005;}
24
25     // Method untuk menampilkan info lagu
26     public void tampilInfo_2005() {
27         System.out.println("- " + judul_2005 + " - " + penyanyi_2005);
28     }
29 }

```

Program di atas merupakan implementasi *Double Linked List* pada sistem playlist musik menggunakan bahasa Java. Program terdiri dari dua kelas yaitu `Lagu_2511532005` sebagai node dan `Musik_2511532005` sebagai pengelola playlist.

Pada program ini setiap lagu disimpan dalam node yang memiliki:

- data judul lagu
- data penyanyi

- pointer next
- pointer prev

Dengan menggunakan *Double Linked List*, playlist dapat ditelusuri dari depan ke belakang maupun dari belakang ke depan.

Kode dan fungsi :

1. judul_2005 = menyimpan judul lagu
2. penyanyi_2005 = menyimpan nama penyanyi
3. next_2005 = pointer ke lagu berikutnya
4. prev_2005 = pointer ke lagu sebelumnya

3. Drive Musik

```

1 package pekan6_2511532005;
2
3 import java.util.Scanner;
4
5 public class Musik_2511532005 {
6     private Lagu_2511532005 head_2005; // Head pointer DLL
7     private Lagu_2511532005 tail_2005; // Tail pointer DLL
8
9     // Constructor
10    public Musik_2511532005() {
11        head_2005 = null;
12        tail_2005 = null;
13    }
14    // 1. tambahLagu_2005() - Tambah di AKHIR playlist (DLL)
15    public void tambahLagu_2005(String judul_2005, String penyanyi_2005) {
16        Lagu_2511532005 laguBaru_2005 = new Lagu_2511532005(judul_2005, penyanyi_2005);
17
18        if (head_2005 == null) {
19            // Playlist kosong
20            head_2005 = tail_2005 = laguBaru_2005;
21        } else {
22            // Tambah di akhir - HUBUNGKAN prev & next
23            tail_2005.setNext_2005(laguBaru_2005);
24            laguBaru_2005.setPrev_2005(tail_2005);
25            tail_2005 = laguBaru_2005;
26        }
27        System.out.println("Lagu berhasil ditambahkan!");
28    }
29    // 2. hapusLaguAwal_2005() - Hapus lagu pertama (head)
30    public void hapusLaguAwal_2005() {
31        if (head_2005 == null) {
32            System.out.println("Playlist kosong! Tidak ada lagu untuk dihapus.");
33            return;
34        }
35        System.out.println("Menghapus: " + head_2005.getJudul_2005());
36        if (head_2005 == tail_2005) {
37            // Hanya 1 lagu
38            head_2005 = tail_2005 = null;
39        } else {

```

```

40 // Update head dan prev pointer
41 head_2005 = head_2005.getNext_2005();
42 head_2005.setPrev_2005(null);
43 }
44 System.out.println("Lagu pertama berhasil dihapus!");
45
46 // 3. tampilMaju_2005() - Traverse dari head ke tail
47 public void tampilMaju_2005() {
48     if (head_2005 == null) {
49         System.out.println("Playlist kosong!");
50         return;
51     }
52     System.out.println("\n=== PLAYLIST (MAJU - HEAD → TAIL) ===");
53     Lagu_2511532005 current_2005 = head_2005;
54     int nomor_2005 = 1;
55     while (current_2005 != null) {
56         System.out.print(nomor_2005 + ". ");
57         current_2005.tampilInfo_2005();
58         current_2005 = current_2005.getNext_2005();
59         nomor_2005++;
60     }
61
62 // 4. tampilMundur_2005() - Traverse dari tail ke head (DLL FITUR WAJIB)
63 public void tampilMundur_2005() {
64     if (tail_2005 == null) {
65         System.out.println("Playlist kosong!");
66         return;
67     }
68     System.out.println("\n=== PLAYLIST (MUNDUR - TAIL → HEAD) ===");
69     Lagu_2511532005 current_2005 = tail_2005;
70     int nomor_2005 = 1;
71     while (current_2005 != null) {
72         System.out.print(nomor_2005 + ". ");
73         current_2005.tampilInfo_2005();
74         current_2005 = current_2005.getPrev_2005();
75         nomor_2005++;
76     }
77
78 // 5. cariLagu_2005() - Search tidak case-sensitive
79 public void cariLagu_2005(String judulCari_2005) {

```

```

79     if (head_2005 == null) {
80         System.out.println("Playlist kosong!");
81         return;
82     }
83
84     Lagu_2511532005 current_2005 = head_2005;
85     boolean ditemukan_2005 = false;
86     int posisi_2005 = 1;
87
88     while (current_2005 != null) {
89         if (current_2005.getJudul_2005().toLowerCase().contains(
90             judulCari_2005.toLowerCase())) {
91             System.out.println("\nLagu ditemukan di posisi " + posisi_2005 + ":");
92             current_2005.tampilInfo_2005();
93             ditemukan_2005 = true;
94         }
95         current_2005 = current_2005.getNext_2005();
96         posisi_2005++;
97     }
98
99     if (!ditemukan_2005) {
100         System.out.println("Lagu " + judulCari_2005 + " tidak ditemukan!");
101     }
102 }
103
104 // MAIN PROGRAM - Driver Class
105 public static void main(String[] args) {
106     Musik_2511532005 playlist_2005 = new Musik_2511532005();
107     Scanner input_2005 = new Scanner(System.in);
108     int pilihan_2005;
109
110     do {
111         // Menu sesuai spesifikasi
112         System.out.println("\n=== Playlist Musik NIM: 2511532005 ===");
113         System.out.println("1. Tambah Lagu");
114         System.out.println("2. Hapus Lagu Pertama");
115         System.out.println("3. Lihat Playlist (Maju)");
116         System.out.println("4. Lihat Playlist (Mundur)");
117         System.out.println("5. Cari Lagu");

```

```

120     pilihan_2005 = input_2005.nextInt();
121     input_2005.nextLine(); // Clear buffer
122
123     switch (pilihan_2005) {
124     case 1:
125         System.out.print("Judul: ");
126         String judul_2005 = input_2005.nextLine();
127         System.out.print("Penyanyi: ");
128         String penyanyi_2005 = input_2005.nextLine();
129         playlist_2005.tambahLagu_2005(judul_2005, penyanyi_2005);
130         break;
131
132     case 2:
133         playlist_2005.hapusLaguAwal_2005();
134         break;
135
136     case 3:
137         playlist_2005.tampilMaju_2005();
138         break;
139
140     case 4:
141         playlist_2005.tampilMundur_2005();
142         break;
143
144     case 5:
145         System.out.print("Cari judul lagu: ");
146         String cari_2005 = input_2005.nextLine();
147         playlist_2005.cariLagu_2005(cari_2005);
148         break;
149
150     case 6:
151         System.out.println("Terima kasih!");
152         break;
153
154     default:
155         System.out.println("Pilihan tidak valid!");
156     }
157     while (pilihan_2005 != 6);
158
159     input_2005.close();
160 }
161 }
162

```

Kode dan fungsi :

1. ☐ head_2005 = menunjuk lagu pertama
2. ☐ tail_2005 = menunjuk lagu terakhir
3. tambahLagu_2005() = Fungsi ini digunakan untuk menambahkan lagu di akhir playlist.
4. hapusLaguAwal_2005() = digunakan untuk menghapus lagu pertama pada playlist.
5. tampilMaju_2005() = digunakan untuk menampilkan playlist dari depan ke belakang menggunakan pointer next.
6. tampilMundur_2005() = digunakan untuk menampilkan playlist dari belakang ke depan menggunakan pointer prev.
7. cariLagu_2005() = digunakan untuk mencari lagu berdasarkan judul.

4. Output

```

=== Playlist Musik NIM: 2511532005 ===
1. Tambah Lagu
2. Hapus Lagu Pertama
3. Lihat Playlist (Maju)
4. Lihat Playlist (Mundur)
5. Cari Lagu
6. Keluar
Pilihan: 1
Judul: reckless
Penyanyi: maddison beer
Lagu berhasil ditambahkan!

=== Playlist Musik NIM: 2511532005 ===
1. Tambah Lagu
2. Hapus Lagu Pertama
3. Lihat Playlist (Maju)
4. Lihat Playlist (Mundur)
5. Cari Lagu
6. Keluar
Pilihan: 1
Judul: bittersweet
Penyanyi: madison beer
Lagu berhasil ditambahkan!

```

```

=== Playlist Musik NIM: 2511532005 ===
1. Tambah Lagu
2. Hapus Lagu Pertama
3. Lihat Playlist (Maju)
4. Lihat Playlist (Mundur)
5. Cari Lagu
6. Keluar
Pilihan: 3

=== PLAYLIST (MAJU - HEAD → TAIL) ===
1. - reckless - maddison beer
2. - bittersweet - madison beer

=== Playlist Musik NIM: 2511532005 ===
1. Tambah Lagu
2. Hapus Lagu Pertama
3. Lihat Playlist (Maju)
4. Lihat Playlist (Mundur)
5. Cari Lagu
6. Keluar
Pilihan: 4

=== PLAYLIST (MUNDUR - TAIL → HEAD) ===
1. - bittersweet - madison beer
2. - reckless - maddison beer

=== Playlist Musik NIM: 2511532005 ===
1. Tambah Lagu
2. Hapus Lagu Pertama
3. Lihat Playlist (Maju)
4. Lihat Playlist (Mundur)
5. Cari Lagu
6. Keluar
Pilihan: 5
Cari judul lagu: bittersweet

Lagu ditemukan di posisi 2:
- bittersweet - madison beer

```

```
=== Playlist Musik NIM: 2511532005 ===  
1. Tambah Lagu  
2. Hapus Lagu Pertama  
3. Lihat Playlist (Maju)  
4. Lihat Playlist (Mundur)  
5. Cari Lagu  
6. Keluar  
Pilihan: 2  
Menghapus: reckless  
Lagu pertama berhasil dihapus!  
  
=== Playlist Musik NIM: 2511532005 ===  
1. Tambah Lagu  
2. Hapus Lagu Pertama  
3. Lihat Playlist (Maju)  
4. Lihat Playlist (Mundur)  
5. Cari Lagu  
6. Keluar  
Pilihan: 6  
Terima kasih!
```